

1. Felder
(4 Punkte)

Implementieren Sie eine Klassenmethode *frequencyOfDigits* in Java, die einen int-Wert *value* (es gilt: *value* ≥ 0) erhält und ein Feld *a* liefert, das die Häufigkeit der Dezimalziffern von *value* enthält.

Beispiel:

geg.: *value* = 1291344561

ges.:

<i>a</i>	0	3	1	1	2	5	1	0	0	1
	0	1	2	3	4	5	6	7	8	9

2. ATG
(4 Punkte)

Gegeben ist ein strukturierter Eingabedatenstrom, der eine Tabelle mit beliebig vielen Zeilen und Spalten beschreibt. Sie können annehmen, dass mindestens eine Zeile und eine Spalte vorhanden sind. Spalten werden mit dem Symbol | getrennt. Zeilen werden mit dem Symbol \\
 getrennt. Eine Zelle kann eine Zahl (number) enthalten, aber auch leer bleiben.

Gesucht ist eine ATG, die überprüft, ob jede Zeile gleich viele Spalten hat und die Anzahl der Zeilen ausgibt.

Beispiel:

Eingabe:

```
17 | 4 | 21 \\  
4 | | 21 \\  
21 | 21 | 42
```

Ausgabe:

Die Tabelle ist gültig.

Anzahl Zeilen: 3

3. OOP
(4 Punkte)

Entwickeln Sie eine Klasse *ArrayQueue*, die das gegebene Interface *Queue* implementiert. Die Elemente werden in einem Feld (Array) gespeichert, dessen Größe *size* beim Konstruktor mitgegeben wird.

```
public interface Queue {  
    void enqueue(Object o); // insert object at the end of the queue  
    Object dequeue();       // remove and return the first object of the queue  
    boolean isEmpty();  
    boolean isFull();  
}
```

Hinweise:

Ist das *ArrayQueue*-Objekt voll, wird bei *enqueue* kein weiteres *Object o* aufgenommen und es wird auch keine Exception geworfen.

Wird *dequeue* für ein leeres *ArrayQueue*-Objekt aufgerufen, wird *null* geliefert und keine Exception geworfen.


```

1) static
public int[] frequencyOfDigits(int value) {
    int[] a = new int[10];
    if (value >= 0) {
        System.out.println("Value has to be greater or exactly zero!");
    } else {
        while (value > 0) {
            int x = value % 10;
            a[x]++;
            value = value / 10;
        }
    }
    return a;
}

```

*
~~if (value == 0) {~~
~~a[0] = 1;~~

2) G(S)

S = Zeile { " \ " Zeile
 Zeile = Spalte { " | " Spalte
 Spalte = (number | " ")

Syntax

sem (2)

1 sem ~~row~~ row++; endsem

2 sem if (lastrow == row) { count++; } else { count = 0; i++; }
~~lastrow = row;~~ endsem

3 sem if (i != count) { System.out.println("Die Tabelle ist ungültig.");
 System.out.println("Anzahl Zeilen: " + i); }
 else { System.out.println("Die Tabelle ist gültig.");
 System.out.println("Anzahl Zeilen: " + i); }
 endsem

4 sem ~~int~~ row = 0; endsem

5 sem ~~int~~ count = 1; int i = 1; ~~lastrow = 0;~~ endsem

6 sem lastrow = row; endsem

%

3) public class ArrayQueue implements Queue {
 private Object[] queue;
 public ArrayQueue() { ~~size~~ ^{size}
 queue = new Object[~~100~~ ^{size}];
~~int size = 100;~~ int size = 100;
 }

@Override
 public void enqueue(Object o) {
 if (!queue.isFull())
~~if (size == queue.length) {~~
 int pos = queue.length;
 queue[~~pos~~ ^{pos}] = o;
~~size++;~~
 }

~~@Override
 public Object dequeue() {
 Object o = queue[0];
 * ~~Object temp~~
 for (int i = 0; i < size; i++) {
 temp = queue[i];
 queue[i] = queue[i+1];
 }
 return o;
 }~~

inefficient
 @Override
 public Object dequeue() {
~~if (!queue.isEmpty())~~
 Object o = queue[0];
 if (queue.isEmpty()) {
 for (int i = 0; i < size; i++) {
 queue[i] = queue[i+1];
 }
 }
 return o;
 }

@Override
 public boolean isEmpty() {
 return queue[0] == null;
 }

@Override
 public boolean isFull() {
 return size == queue.length;
 }

* if (!queue.isEmpty()) {